INDEX

NAME YOGASRI S SUBJECT

STD. DIV. ROLL NO. SCHOOL

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
1.	16/7/26	Study of various network commands Linux and windows		
2.	09/7/26	Experiments on cisco packet tracer (simulation tools):		
3.	25/7/26	study of network cables		
4.	30/7/26	LAN using a switch the Ethernet cables.		
5.	1/8/26	Experiments on packet capture tool		
6.	6/8/26	Error detection and correction using Hamming code		
7.	8/8/26	Sliding window protocol		
8a.	13/8/26	Live Hot discovery using NMAP scans		
8b.	13/8/26	wireless lan configuration		
9.	20/8/26	Implementation of subnetting		
10a.	22/8/26	Internetworking with router		
10b.	27/8/26	Internetworking with Router, DHCP server, and Internet cloud		

			DATE	TEACH
11.a)	29/8/26	Simulate static routing configuration		
11.b)	10/9/26	simulate RIP using cisco packet Tracer		
12.a)	12/9/26	Echo client server using TCP sockets		
12.b)	17/9/26	client chat server using TCP sockets		
13	24/9/26	Implement your own ping program		
14.	26/9/26	Network Interface information in java		
15	8/10/26	web log analisis using web server		

Completed

Experiment : 1

16/7/26

Study of various network commands Linux and windows

Windows commands

1. arp - Address resolution protocal
2. hostname - computer name
3. ip config - computer requirments
4. nbtstat - network bios
5. netstat - network statics
6. nslookup - name server
7. pathping - combination of ping and tracert commands
8. ping - package internet groper
9. route - Manipulate the IP routing table

arp

```
C:\Users\fcs>arp

Displays and modifies the IP-to-Physical address translation tables used by
address resolution protocol (ARP).

ARP -s inet_addr eth_addr [if_addr]
ARP -d inet_addr [if_addr]
ARP -a [inet_addr] [-N if_addr] [-v]

  -a            Displays current ARP entries by interrogating the current
                protocol data.  If inet_addr is specified, the IP and Physical
                addresses for only the specified computer are displayed.  If
                more than one network interface uses ARP, entries for each ARP
                table are displayed.
  -g            Same as -a.
  -v            Displays current ARP entries in verbose mode.  All invalid
                entries and entries on the loop-back interface will be shown.
  inet_addr     Specifies an internet address.
  -N if_addr    Displays the ARP entries for the network interface specified
                by if_addr.
  -d            Deletes the host specified by inet_addr. inet_addr may be
                wildcarded with * to delete all hosts.
  -s            Adds the host and associates the Internet address inet_addr
                with the Physical address eth_addr.  The Physical address is
                given as 6 hexadecimal bytes separated by hyphens.  The entry
                is permanent.
  eth_addr      Specifies a physical address.
  if_addr       If present, this specifies the Internet address of the
                interface whose address translation table should be modified.
                If not present, the first applicable interface will be used.
Example:
  > arp -s 157.55.85.212   00-aa-00-62-c6-09  .... Adds a static entry.
  > arp -a                                    .... Displays the arp table.
```

hostname

```
C:\Users\fcs>hostname
DESKTOP-G0LVE4J
```

ipconfig

```
C:\Users\Tcs>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::98bf:3a79:be3d:10b0%14
   IPv4 Address. . . . . . . . . . . : 172.16.10.113
   Subnet Mask . . . . . . . . . . . : 255.255.252.0
   Default Gateway . . . . . . . . . : 172.16.8.1
```

nbtstat

```
C:\Users\Tcs>nbtstat

Displays protocol statistics and current TCP/IP connections using NBT
(NetBIOS over TCP/IP).

NBTSTAT [ [-a RemoteName] [-A IP address] [-c] [-n]
        [-r] [-R] [-RR] [-s] [-S] [interval] ]

  -a   (adapter status) Lists the remote machine's name table given its name
  -A   (adapter status) Lists the remote machine's name table given its
                        IP address.
  -c   (cache)          Lists NBT's cache of remote [machine] names and their IP addresses
  -n   (names)          Lists local NetBIOS names.
  -r   (resolved)       Lists names resolved by broadcast and via WINS
  -R   (Reload)         Purges and reloads the remote cache name table
  -S   (Sessions)       Lists sessions table with the destination IP addresses
  -s   (sessions)       Lists sessions table converting destination IP
                        addresses to computer NETBIOS names.
  -RR  (ReleaseRefresh) Sends Name Release packets to WINS and then, starts Refresh

  RemoteName   Remote host machine name.
  IP address   Dotted decimal representation of the IP address.
  interval     Redisplays selected statistics, pausing interval seconds
               between each display. Press Ctrl+C to stop redisplaying
               statistics.
```

netstat

```
C:\Users\Tcs>netstat

Active Connections

  Proto  Local Address          Foreign Address        State
  TCP    172.16.10.113:50979    172.16.8.1:domain      ESTABLISHED
  TCP    172.16.10.113:60031    4.213.25.242:https     ESTABLISHED
  TCP    172.16.10.113:61835    sf-in-f188:5228        ESTABLISHED
  TCP    172.16.10.113:61909    4.213.25.240:https     ESTABLISHED
  TCP    172.16.10.113:61915    123:http               TIME_WAIT
  TCP    172.16.10.113:61917    lcmaaa-au-in-f14:https ESTABLISHED
  TCP    172.16.10.113:61918    lcmaaa-au-in-f14:https TIME_WAIT
  TCP    172.16.10.113:61919    142.251.150.4:https    ESTABLISHED
  TCP    172.16.10.113:61920    142.251.150.4:https    ESTABLISHED
  TCP    172.16.10.113:61921    20.42.65.88:https      TIME_WAIT
  TCP    172.16.10.113:61922    sg02s06-in-f4:https    ESTABLISHED
  TCP    172.16.10.113:61923    172.217.111.4:https    ESTABLISHED
  TCP    172.16.10.113:61924    123:http               TIME_WAIT
  TCP    172.16.10.113:61925    104.18.12.47:https     ESTABLISHED
  TCP    172.16.10.113:61926    172.217.177.4:https    ESTABLISHED
  TCP    172.16.10.113:61927    150.171.110.85:https   ESTABLISHED
  TCP    172.16.10.113:61928    lchomp-in-f84:https    ESTABLISHED
  TCP    172.16.10.113:61929    lchomp-in-f84:https    ESTABLISHED
  TCP    172.16.10.113:61930    lchomp-in-f84:https    ESTABLISHED
  TCP    172.16.10.113:61931    [illegible]:https      ESTABLISHED
  TCP    172.16.10.113:61932    423:202 229:22:https   ESTABLISHED
  TCP    172.16.10.113:61936    57.155.48.210:https    ESTABLISHED
  TCP    172.16.10.113:61937    123:http               ESTABLISHED
  TCP    [fe80::98bf:3a79:be3d:10b0%14]:1521  DESKTOP-G0EVE43:49674  ESTABLISHED
  TCP    [fe80::98bf:3a79:be3d:10b0%14]:49674  DESKTOP-G0EVE43:1521   ESTABLISHED
```

lookup

```
C:\Users\Tcs>nslookup google.com
Server:  UnKnown
Address:  172.16.8.1

Non-authoritative answer:
Name:    google.com
Addresses:  2404:6800:4007:808::200e
          142.250.77.110
```

pathping

```
C:\Users\Tcs>pathping

Usage: pathping [-g host-list] [-h maximum_hops] [-i address] [-n]
                [-p period] [-q num_queries] [-w timeout]
                [-4] [-6] target_name

Options:
  -g host-list     Loose source route along host-list.
  -h maximum_hops  Maximum number of hops to search for target.
  -i address       Use the specified source address.
  -n               Do not resolve addresses to hostnames.
  -p period        Wait period milliseconds between pings.
  -q num_queries   Number of queries per hop.
  -w timeout       Wait timeout milliseconds for each reply.
  -4               Force using IPv4.
  -6               Force using IPv6.
```

ping

```
C:\Users\Tcs>ping

Usage: ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS]
            [-r count] [-s count] [[-j host-list] | [-k host-list]]
            [-w timeout] [-R] [-S srcaddr] [-c compartment] [-p]
            [-4] [-6] target_name

Options:
    -t             Ping the specified host until stopped.
                   To see statistics and continue - type Control-Break;
                   To stop - type Control-C.
    -a             Resolve addresses to hostnames.
    -n count       Number of echo requests to send.
    -l size        Send buffer size.
    -f             Set Don't Fragment flag in packet (IPv4-only).
    -i TTL         Time To Live.
    -v TOS         Type Of Service (IPv4-only. This setting has been deprecated
                   and has no effect on the type of service field in the IP
                   Header).
    -r count       Record route for count hops (IPv4-only).
    -s count       Timestamp for count hops (IPv4-only).
    -j host-list   Loose source route along host-list (IPv4-only).
    -k host-list   Strict source route along host-list (IPv4-only).
    -w timeout     Timeout in milliseconds to wait for each reply.
    -R             Use routing header to test reverse route also (IPv6-only).
                   Per RFC 5095 the use of this routing header has been
                   deprecated. Some systems may drop echo requests if
                   this header is used.
    -S srcaddr     Source address to use.
    -c compartment Routing compartment identifier.
    -p             Ping a Hyper-V Network Virtualization provider address.
    -4             Force using IPv4.
    -6             Force using IPv6.
```

route





```
C:\Users\Jorge>route

Manipulates network routing tables.

ROUTE [-f] [-p] [-4|-6] command [destination]
                  [MASK netmask]  [gateway] [METRIC metric]  [IF interface]

  -f            Clears the routing tables of all gateway entries.  If this is
                used in conjunction with one of the commands, the tables are
                cleared prior to running the command.

  -p            When used with the ADD command, makes a route persistent across
                boots of the system. By default, routes are not preserved
                when the system is restarted. Ignored for all other commands,
                which always affect the appropriate persistent routes.

  -4            Force using IPv4.

  -6            Force using IPv6.

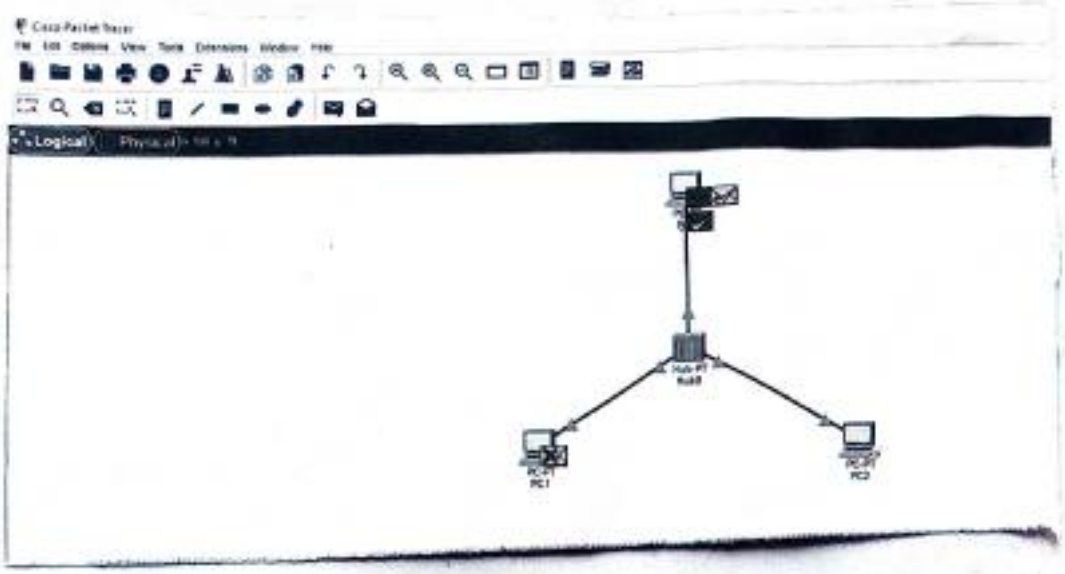
  command       One of these:
                  PRINT     Prints a route
                  ADD       Adds   a route
                  DELETE    Deletes a route
                  CHANGE    Modifies an existing route
  destination   Specifies the host.
  MASK          Specifies that the next parameter is the 'netmask' value.
  netmask       Specifies a subnet mask value for this route entry.
                If not specified, it defaults to 255.255.255.255.
  gateway       Specifies gateway.
  interface     the interface number for the specified route.
  METRIC        specifies the metric, ie. cost for the destination.

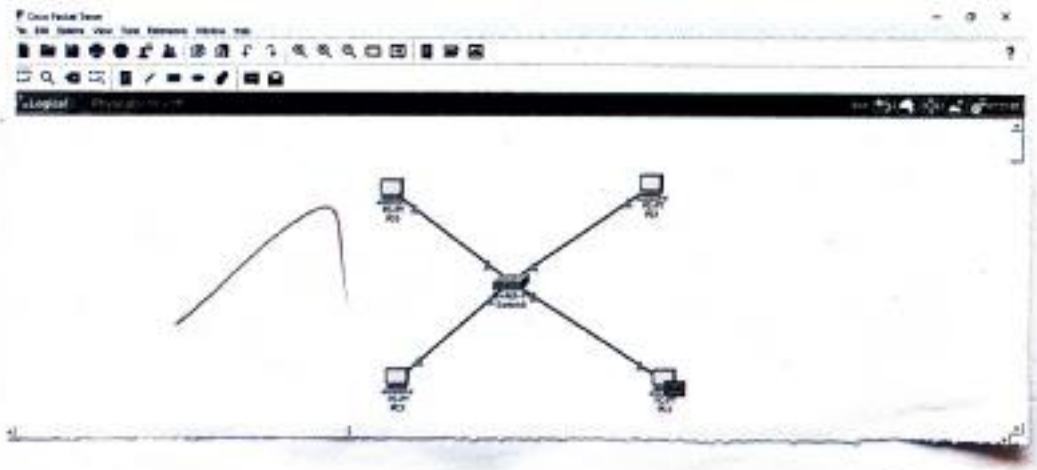
All symbolic names used for destination are looked up in the network database
file NETWORKS. The symbolic names for gateway are looked up in the host name
database file HOSTS.

If the command is PRINT or DELETE. Destination or gateway can be a wildcard,
(wildcard is specified as a star '*'), or the gateway argument may be omitted.
```

Result :

Thus the study of various network commands linux and windows is successfully completed.





Ex:NO
:3
28/7/26

Cisco Packet Tracer & Network Devices Observation

Aim:

To study the cisco packet tracer installation. explore its user interface overview, and analyze the following behaviour of Hub & switch using simple network.

objectives:

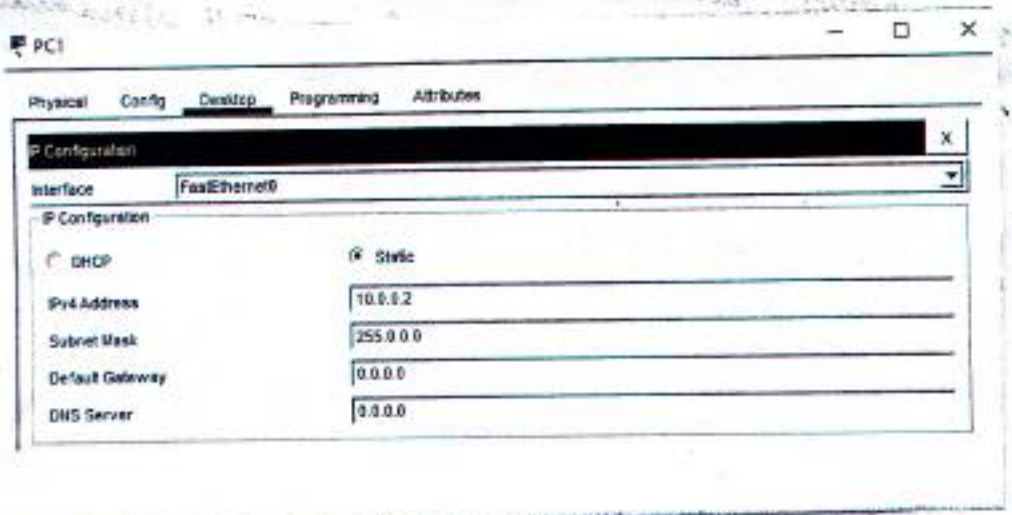
* to get familiarized with the cisco packet Tracer simulation environment and workspace
* To construct star topology networks using a Hub and a switch
* To observe, test, and compare the packet forwarding behaviour of layer 1 (Hub) and layer 2 (switch) devices using PDU simulation.

Procedure:

A. setting up the network topologies

1. open cisco packet Tracer.
2. From the network component box, drag and drop
   - L4 4 Generic PCs and Hub
   - L1 4 Generic PCs and 1 switch
3. select connections. copper straight - through cable.
4. connect each PC to the Hub using the cable. Ensures the link LED turns green


PC0
Physical
Config
Desktop
Programming
Attributes
IP Configuration
Interface
FastEthernet0
IP Configuration
DHCP
Static
IPv4 Address
10.0.0.1
Subnet Mask
255.0.0.0
Default Gateway
0.0.0.0
DNS Server
0.0.0.0


PC1
Physical
Config
Desktop
Programming
Attributes
IP Configuration
Interface
FastEthernet0
IP Configuration
DHCP
Static
IPv4 Address
10.0.0.2
Subnet Mask
255.0.0.0
Default Gateway
0.0.0.0
DNS Server
0.0.0.0

B. IP configuration

1. Click on each PC → Go to the Desktop tab → click IP configuration.
2. Assign static IP addresses and subnet masks within the same networks range.

C. Testing packing Transmission & Behaviour analysis.

1. Select the add simple PDU tool from the right toolbar.
2. Drop the PDU on a source and a destination connected to the Hub.
3. Switch to simulation mode and click play to observe the packet flow.
4. Repeat the process for PCs connected to the switch.

Hub behaviour:

* Boardcasting: when a PDU is sent from one PC to another via the Hub, the Hub services the frame on one port and retransmits it out to all other connected ports.
* Frame rejection: The destination PC accepts the frame, while non-destination PCs reject it
* Layer & Domain: Hub operates at layer 1 and shares a single collision domain and be broadcast

switch behaviour:

* unicasting: when a PDU is sent via the switch, the switch inspects the destination MAC address in frame header.

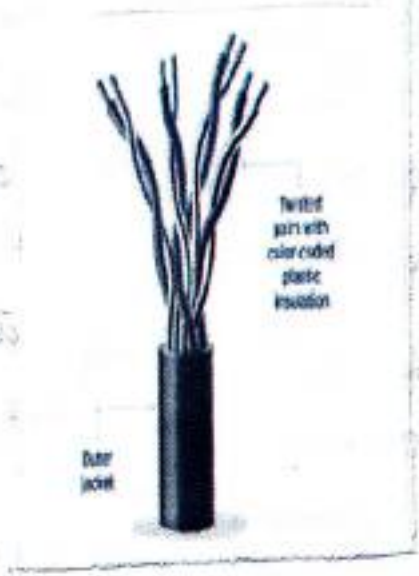
* Target forwarding: using its MAC address table, the switch forwards the packet only to the specific port connected to the intended destination PC. other PCs do not receive the packet.

b) college network topology:

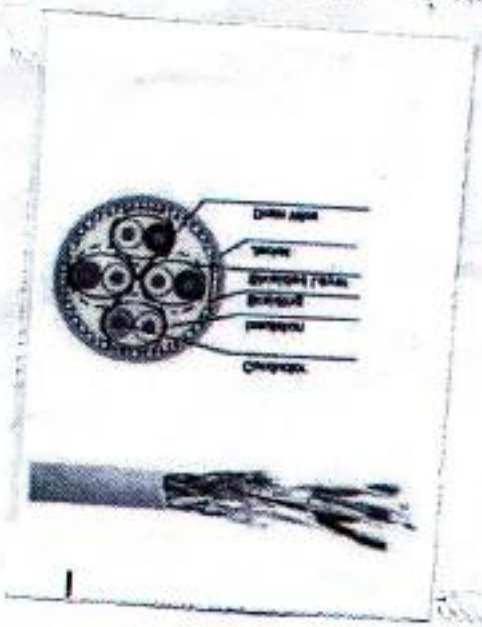
topology implemented: star topology

Result:

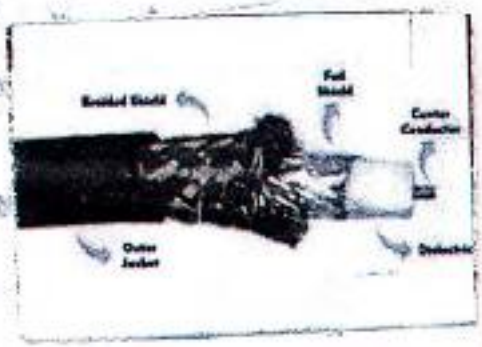
1. The cisco packet Tracer workspace and Tool space were successfully studied
2. The behavior of the hub and switch were observed and documented.
3. The college laboratory network topology was identified as a star topology


Twisted pairs with color-coded plastic insulation
Outer jacket


Shielded twisted pair







Ex No: 2

25/7/25

2 STUDY OF NETWORK CABLES

Aim:

Study of different type of network cables.

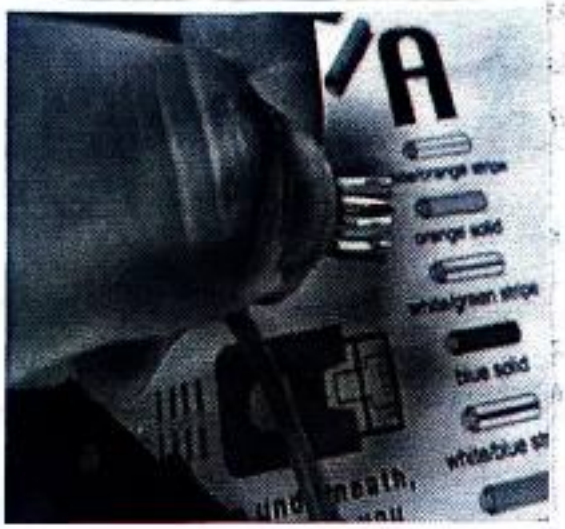
a) understanding different types of cables:

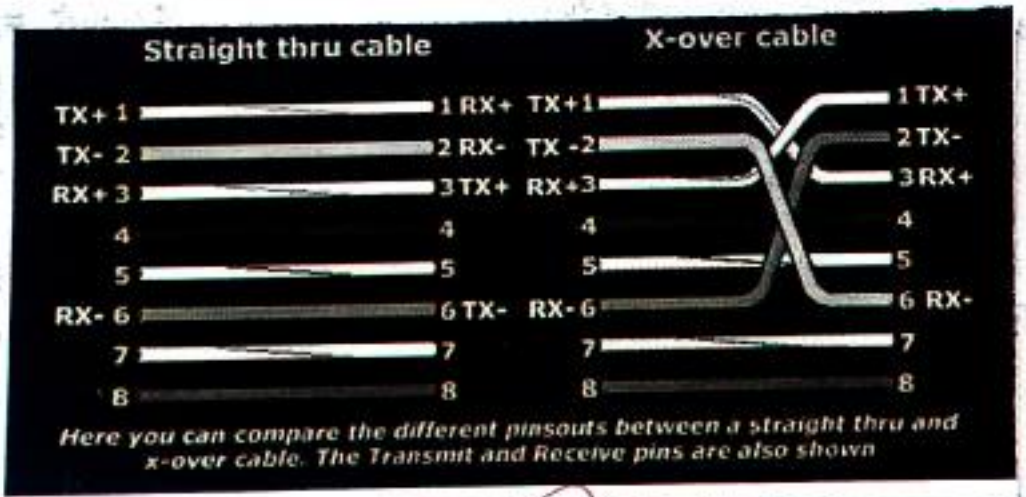
Different types of cables used in networking are:

1. unshielded twisted pair (UTP) cable
2. shielded twisted pair cable (STP)
3. coaxial cable
4. Fiber optic cable.

cable Type	category	Maximum Data Transmission	Advantages / Disadvantages	Application / use
UTP	category 3	10 bps	Adv: * cheaper in cost * Easy to install and they have a smaller overall diameter Disadv: * More prone to (EMI) Electromegnetic Interface and noise	10-Base-T Ethernet
	category 5	up to 100 Mbps		Fast Ethernet Gigabit Ethernet
	category 5e	1Gbps		Fast Ethernet Gigabit Ethernet
STP	category 6, 6a	10Gbps	Adv: * shielded * Faster than UTP * Less susceptible to noise and interference Disadv: * Expensive * Greater installation effort.	Gigabit Ethernet, 10G Ethernet (55m) widely used in data centres
STP	category 7	10Gbps		Gigabit Ethernet, 10G Ethernet (100m)
Coaxial cable	RG-6 RG-59 RG-11	10-100 Mbps	Adv: * High Bandwidth * Immune to interference * Low loss bandwidth * Versatile	speed of signal is 800m television network High speed internet connections







Here you can compare the different pinsouts between a straight thru and x-over cable. The Transmit and Receive pins are also shown

Fiber optics cable	single mode multi mode	100Gbps	Adv: * High speed * High bandwidth * High security * long distance. Disadv: * Expensive * Requires skilled installer	Maximum distance of fiber optic cable is arround 100 meters

b) Make your own Ethernet cross-over cable / straight cable :

Tools and parts needed.

* Ethernet cabling . CAT5e is certified for gigabit support , but CAT5 cabling works as well just over shorter distances.
* A crimping tool : This is an all-in-one networking tool shaped to push down the pins in the plug and strip and cut the shidding of the cables.
* Two RJ45 plugs
* optional two plug shields.

procedure:

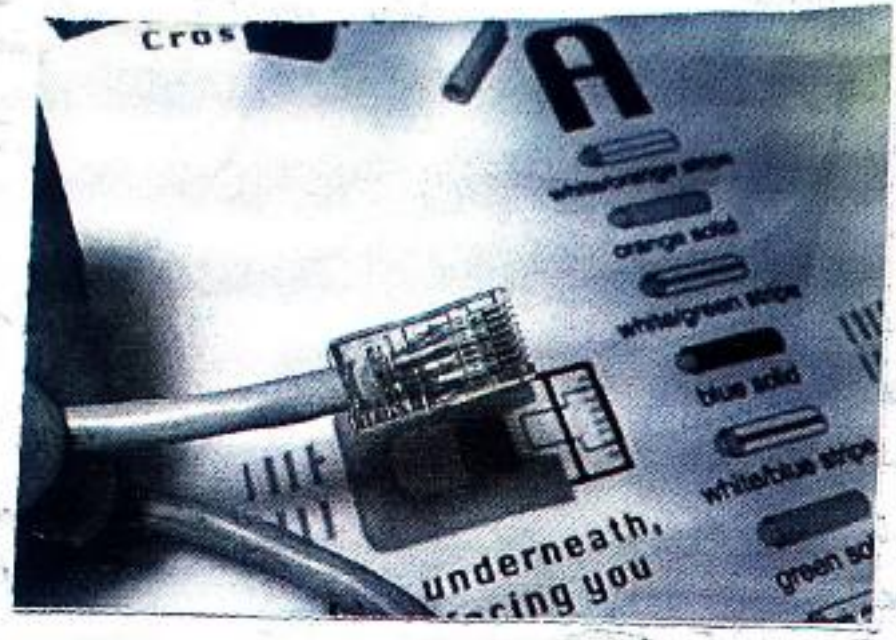
step 1 : Insert the cable through the cable root

step 2 : Remove about 1.5 cm of the outer cable cover using a cripping tool.

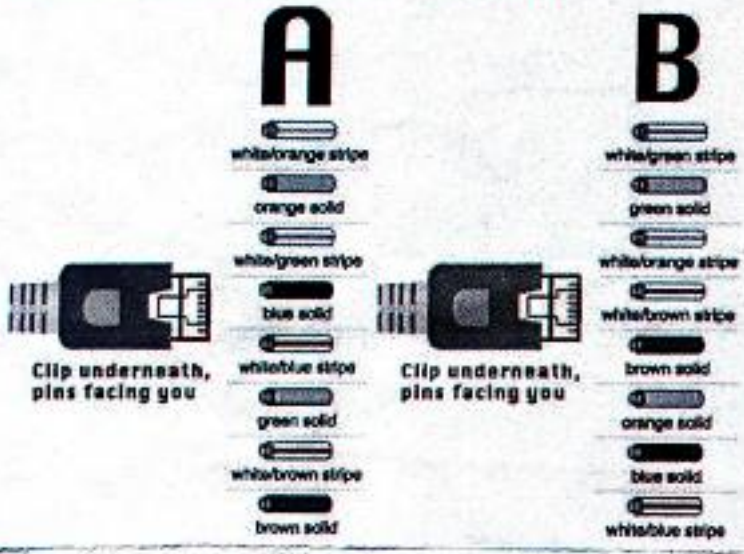
step 3 : seperate the 4 twisted wire pairs and arrange the wire colors in the correct order.

step 4 : straighten the wires , trim them to equal length , an insert them carefully into the RJ45 connectors.

step 5 : Ensure the cable jacket reaches inside the RJ45 connector.



Straight through network cable: both sides should be A
Crossover cable: One side A, one side B



step6 : Place the RJ45 connector into the crimping tool and press firmly to fix the connector.

step7 : Repeat the process of the other end.

step8 : Test the cable using a cable tester or by connecting two devices.

Result :

The twisted pair cable was successfully prepared using RJ45 connector. Tested with a cable tester, cable Tester, and verified to provide proper network connectivity.

Ex: No: 4
30/7/26

Setup and Configuration a LAN using a switch and Ethernet cables.

Aim:

To set up and configure a Local Area Network (LAN) using a switch and Ethernet cables.

What is a LAN?

A Local Area Network (LAN) is a network that connects computers and other devices within a small area such as a home, school, office or laboratory. It allows users to share files, printers, and internet access and enables communication b/w connected devices.

How to set up a LAN:

step 1:
plan the network arrange 4 PCs, 1 switch, and 4 Ethernet cables.

step 2:
connect each PC to the switch using Ethernet cables.

step 3:
Assign IP addresses to all PCs:

* PC1 : 10.1.1.1, subnet Mask : 255.0.0.0
* PC2 : 10.1.1.2, subnet Mask : 255.0.0.0
* PC3 : 10.1.1.3, subnet Mask : 255.0.0.0
* PC4 : 10.1.1.4, subnet Mask : 255.0.0.0

step 4:
configure the switch and assign:

1) switch IP : 10.1.1.5
2) subnet Mask : 255.0.0.0

step 5:

check network connectivity using the ping command.

step 6:

share a folder:

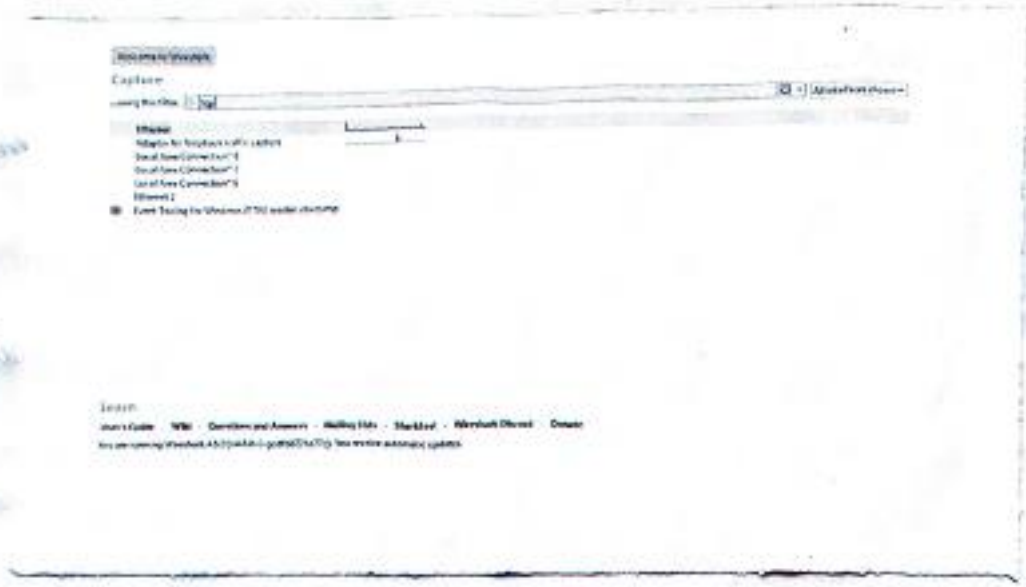
- Right - click the folder - properties - sharing - share with Everyone.

step 7:

Access the shared folder from another computer on the LAN.

Result:

The Local Area Network (LAN) was successfully configured using a switch and Ethernet cables. All computers communicated successfully using the ping command, and file sharing was verified across the network.


Capture
Ethernet
Learn
User's Guide
Wiki
Questions and Answers
Mailing Lists
Sharkfest
Wireshark Discord
Donate



Ex.No :5

18/9

Experiments on Packet Capture Tool

Aim:

To capture, filter, view and analyze network packets from the an Ethernet interface using the wireshark packet sniffer tool.

objectives:

1. To understand the structure, and functioning of packet sniffers like wireshark and tcpdump.
2. To capture active network traffic using promiscuous mode on a local interface.
3. To apply display filters for specific network protocols including TCP, UDP, ARP, DNS, HTTP, ICMP and DHCP.
4. To analyze packet structures
5. To inspect TCP flow graphs and full packet stream conversations.

Apparatus / Software Required:

* system : computer with windows / Linux / macOS
* Software: Wireshark Network Analyzer
* Network Interface: Active Ethernet LAN or wifi interface.

procedure:

General Packet Capture Procedure

1. Launch wireshark on your system
2. select the active Local Area Connection / Wi-fi interface.
3. Go to capture → options and select "stop capture automatically after 100 packets".



4. Click start capture

5. After capturing 100 packets, save the captured file for analysis.

protocal Filtering procedures:

1. TCP / UDP packets:

* Enter tcp or udp in the wireshark filter bar
* To view the communication flow, click statistics → Flow Graph.

2. ARP Packets:

* Enter arp in the filter search bar to display Address resolution protocol packets.
* Inspect the hardware and protocol addresses in the packet Details pane.

3. DNS packets:

* Enter DNS in the filter search bar to capture domain name system queries and responses.
* view the conversation flow via statistics → flow Graph.

4. HTTP packets:

* Enter http in the display filter book.
* Inspect web request and response header details.

5. IP / ICMP Packets:

* Enter icmp or ip in the filter bar to observe ping / echo traffic

6. DHCP packets:

* Enter dhcp in the filter bar to view dynamic IP address assignments.

Result:

1. The wireshark packet capturing tool was successfully configured and operated.
2. Network packets were captured across the ethernet interface, and specific protocols were isolated and analyzed using display filters.
3. All student observation questions regarding network protocols, transport layers, and address types were answered accurately.

Ex No :6
6/8/24

ERROR DETECTION and CORRELATION USING HAMMING CODE

Aim:

To write a program to implement error detection and correlation using the Hamming code.

Algorithm:

1. Start the program
2. Read the number 4 bit data blocks (n) forming the input stream.
3. For each 4-bit data block D = d_1 d_2 d_3 d_4, perform Hamming encoding.
   a. Arrange 7 bits with using even parity at positions 1, 2, and 4. and data bits at positions 3, 5, 6 and 7.
   b. compute the parity bit using even parity
   c. From the 7 bit Hamming code word p_1 p_2 d_1 p_4 d_2 d_3 d_4
4. simulate transmission by introducing a single-bit error at a choosen bit position in the code word.
5. At the receiver recompute the check bits from the received code word.
6. Compute error position = c_4 c_2 c_1
7. If error position = 0, no error has occured.
8. Extract the original 4 data bits from the corrected 7-bit code word.
9. Repeat steps 3 to 8 for all n blocks in the data stream.
10. stop the program.

=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===

Enter number of 4-bit data blocks in the stream

--- Block 1 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
Original data bits : 1011
Encoded Hamming code : 0110011
Enter bit position to introduce error (1-7, 0 = no error): 3
Transmitted code (error at position 3): 0100011
Received code : 0100011
checking for errors...
Error detected at bit position: 3
correcting error...
corrected code : 0110011
Received data bits : 1011

--- Block 2 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 0 1 1 0
original data bits : 0110
Encoded Hamming code : 1100110
Enter bit position to introduce error (1-7, 0 = no error): 6
Transmitted code (error at position 6): 1100100
Received code : 1100100
checking for errors...
Error detected at bit position: 6
correcting error...
corrected code : 1100110
Recovered data bits : 0110

```
def encode(d):
    h = [0] * 7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]
    return h
def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
    error_pos = c1*1 + c2*2 + c4*4
    if error_pos != 0:
        print(f"Error detected at bit position: {error_pos}")
        print("Error Correcting error...")
        h[error_pos - 1] ^= 1
    else:
        print("No error Detected.")
    return error_pos
def to_str(bits):
    return "".join(str(b) for b in bits)
def main():
    print("=== Hamming code - Error Detection and Correction ===\n")
    n = int(input("Enter number of 4-bit data blocks in the stream: "))
    for b in range(1, n+1):
        print(f"\n--- Block {b} ---")
```

```
d = list(map(int, input("Enter 4 data bits ").
                                    split()))
h = encode(d)
print("Original data bits  :", to_str(d))
print("Encoded Hamming Code :", to_str(h))
pos = int(input(
    "Enter bit position to introduce error (1-7,
                    0=no error): "))
if 1 <= pos <= 7:
    h[pos-1] ^= 1
    print(f"Transmitted Code {pos}:",
        to_str(h)
    )
else:
    print("Transmitted Code :", to_str(h))
print("Received Code  :", to_str(h))
print("checking for errors...")
defect_and_correct(h)
print("Corrected Code :", to_str(h))
rd = [h[2], h[4], h[5], h[6]]
print("Recovered Data bits  :", to_str(rd))

if __name__ == "__main__":
    main()
```

Result:

The program to implement error detection and correction using the Hamming code concept was successfully written in python, executed and verified using the sample input data stream.

Exp: No: 7 SLIDING WINDOW PROTOCOL

8/8/16

Aim:

write a program to implement flow control at data link layer using sliding window protocol. simulate the flow frames from the node to another.

Algorithm:

1. start the program
2. Enter the window size
3. Enter the text message.
4. Divide the message into frames, one character per frame.
5. store the frames in sender_Buffer
6. send the frames to the receiver
7. Receiver checks the frame number
8. If the frame is correct, send ACK
9. If the is wrong, send NACK
10. sender reads Receiver_Buffer.
11. If ACK is received, move the window and send the next frames.
12. If NACK is received, move the window and send the next frames.
13. Repeat until all frames are sent successfully.
14. stop the program.

python Code:

```
import time
import os
SENDER_FILE = "Sender_Buffer.txt"
RECEIVER_FILE = "Receiver_Buffer.txt"
def create_frames(message, start, window_size):
```

```
    frames = []
    end = min(start + window_size, len(message))
    for i in range(start, end):
        frames.append(f"Frame No: {i+1}, DATA:
                      {message[i]}")
    return frames
    def write_sender_buffer(frames):
        with open(SENDER_FILE, "w") as file:
            for frame in frame:
                file.write(frame + "\n")

def read_sender_buffer():
    with open(SENDER_FILE, "r") as file:
        return file.readlines()

def write_receiver_buffer(ack):
    with open(RECEIVER_FILE, "w") as file:
    file.write(f"ACK: {ack}\n")
def read_receiver_buffer():
    with open(RECEIVER_FILE, "r") as file:
    data = file.read().strip()
    if data.startswith("ACK:"):
        return int(data.split(":")[1].strip())
    return -1
def receiver(expected_frame):
    print("Receiver Received ->", line.strip())
    frame_no = int(line.split(",")[0].split(":")[1].strip())
    if frame_no == expected_frame:
        print(f"Frame {frame_no} received correctly.")
        write_receiver_buffer(frame_no)
        print(f"Receiver send ACK: {frame_no}")
        return frame_no
    else
        print(
            f"Expected Frame {expected_frame}."
            f" but received Frame {frame_no}")
```

Output :

Message : HELLO
Window size : 3
==========
Sender Buffer

Frame No : 1, Data : H
Frame No : 0, Data : E
Frame No : 3, Data : L
Frames send successfully
waiting for acknowledgement ...
Receiver is reading Sender_Buffer ...
Receiver_received -> Frame No : 1 DATA - H
Frame 1 received correctly.
Receiver sent ACK : 1
Sender is reading Receiver_Buffer ...
Received ACK : 1
ACK 1 is correct.
sliding window moved. Next frame = 0
==========
Sender Buffer

Frame No : 2, DATA : E
Frame No : 3, DATA : L
Frame No : 4, DATA : L
Frames sent successfully.

```
    write_receiver_buffer(expected_frame-1)
    print(f"Receiver sent NACK: {expected_frame-1}")
    return expected_frame-1
def sender(message, window_size):
    next_frame=0
    while next_frame < len(message)
    frames = create_frames(message, next_frame, window_size)
    write_sender_buffer(frames)
    print("\n ==========")
    print("Sender Buffer")
    for frame in frames:
        print(frame)
    print("\n Frames sent successfully.")
    print("waiting for acknowledgement...")
    time.sleep(2)
    expected_frame = next_frame+1
    ack = receiver(expected_frame)
    print("\n Sender is reading Receiver_Buffer...")
    received_ack = read_receiver_buffer()
    print("Received ACK:", received_ack)
    if received_ack >= expected_frame:
        print(
            f"ACK {received_ack} is correct."
        )
        next_frame = received_ack
        print( f"sliding window moved." f"Next frame
                = {next_frame+1}" )
    else:
        print("NACK received.")
        print("Resending old frames...")
        time.sleep(2)
        print("\n Resending frames:")
        for frame in frames:
            print(frame)
        ack = receiver( expected_frame)
        received_ack = read_receiver_buffer()
        if received_ack = expected_frame:
            next_frame = received_ack
        else:
            print("Error still exists. Retrying...")
```

```
print("==========")
print("All frames transmitted successfully!")
print("==========")
def main():
    print("**********")
    print("SLIDING WINDOW PROTOCOL")
    print("==========")
    window_size = int(input("Enter window size:"))
    message = input("Enter Text Message:")
    open(SENDER_FILE, "w").close
    open(RECEIVER_FILE, "w").close
    sender(message, window_size)
    print("\n Sender Buffer file:", os.path.abspath(
                                    SENDER_FILE))
    print("Receiver Buffer file:", os.path.abspath(
                                    RECEIVER_FILE))
if __name__ == '__main__':
    main()
```

Result:

Thus the sliding window protocol was successfully implement using python. Frame were transmitted from sender to receiver, and ACK/NACK was used for error detection and retransmission.

Ex. NO: 8(a)

Date: 13/8/26

LIVE HOST DISCOVERY USING NMAP SCANS

Aim:

To discover live hosts on a network using various nmap host discovery scans on the TryHackMe platform.

procedure:

1. connect the TryHackMe network using the openVPN configuration file (or) start the browser based AttackBox.
2. Define Target hosts using one of the following.
   * Individual IPs / Hostnames: MACHINE_IP
   * IP Range: 10.11.12.15-20
   * subnet (CIDR): 10.10.12.0/24
3. ARP (SCAN)
   - use an ARP scan when the Target is on the same local Ethernet subnet to discover MAC addresses without performing ports scans.
4. ICMP scans:
   - ICMP Echo ping:
   - ICMP Timestamp ping
   - ICMP Address Mask ping.
5. TCP ping scans:
   - Target port 22
   - Target fault port 80
   - TCP ACK ping
6. UDP ping scan:
   - send UDP packets to closed ports to checks for ICMP port unreachable replies.

7. Reverse - DNS Lookup:

perform reverse DNS queries on all potential all hosts within a subnet to gather hostname insights.



Result:

The live hosts on the target subnet were successfully identified using Nmap, ARP, ICMP, TCP and UDP host discovery scanning techniques. The behaviour of each protocol was verified and recorded.

Ex No: 8b
13/8/26

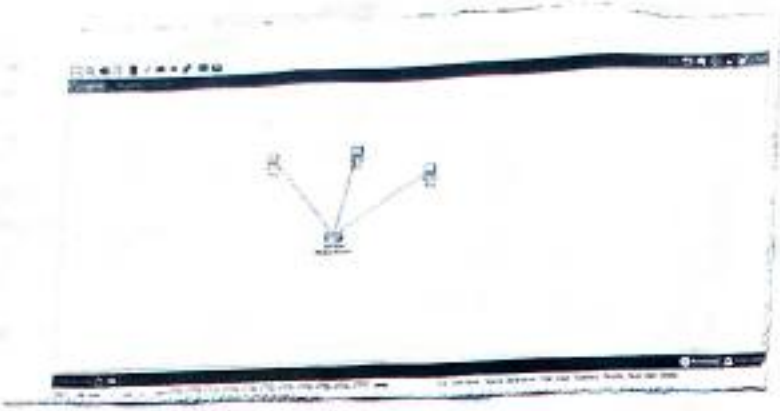
Wireless LAN Configuration

Aim:

To configure a wireless LAN (WLAN) connecting 3PCs to a Linksys wireless Router with static IPs and WEP security in Cisco Packet Tracer.

Procedure:

1. Topology setup
   - * place 1 Linksys wireless Router and 3PCs.
   - * Turn off each PC, replace the Ethernet module with a wireless Module and turn the PC back on.
2. Router setup
   - * Open Router GUI → Administration Tab → setup username & password to admin → click save setting.
   - * Go to wireless Tab → set SSID to MotherNetwork
3. static IP Assignment
   - * Router : 192.168.0.1
   - * PC0 : IP 192.168.0.2 | Subnet 255.255.255.0 | Gateway 192.168.0.1.
   - * PC1 : IP 192.168.0.3 | subnet 255.255.255.0 | Gateway 192.168.0.1.
   - * PC2 : IP 192.168.0.4 | subnet 255.255.255.0 | Gateway 192.168.0.1
4. wireless Connection :
   - * On each PC, open Desktop → PC wireless → connect tab → click refresh.
   - * select MotherNetwork, click network, enter WEP





Result:

The wireless LAN topology was designed. secured using WEP key, and all PCs successfully connected to Mothernetwork.

Ex. No: 9

Implementation of subnetting

20/8/26

Aim:

To implement classless IP subnetting using routers, switches, and PCs in Cisco Packet Tracer and verify connectivity.

procedure:

1. Topology setup:
   - ↳ place 2 Routers (R_1, R_0), 2 switches (S_1, S_0) and PCs (PC1, PC5 for each subnet)
   - ↳ connect routers to switches, and switches to PCs using Ethernet cables.
2. subnet calculation (/27 Mask):
   - ↳ subnet mask: 255.255.255.224 (/27)
   - ↳ subnet 1: 192.168.1.0/27 (Gateway: 192.168.1.1)
3. Router Configuration (R_1 Example):
   - ↳ open router CLI and Type

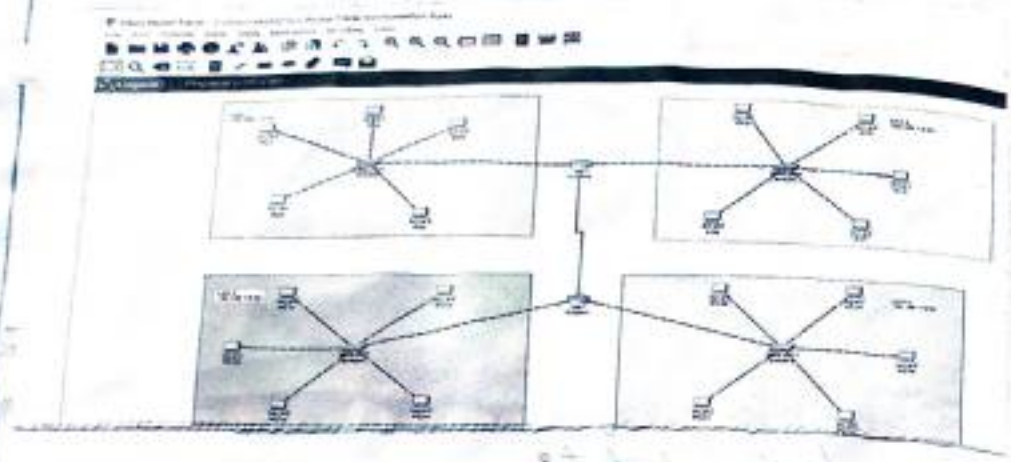
```
enable
configure terminal
interface GigabitEthernet 0/0
ip address 192.168.1.1 255.255.255.224
no shut down
exit
```

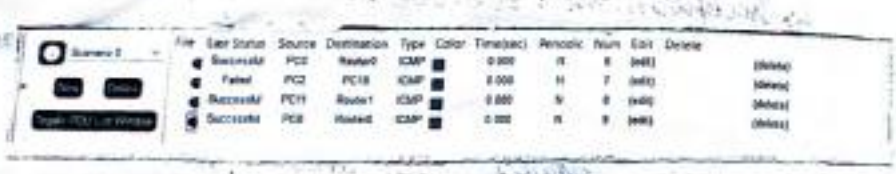
4. PC Configuration:
   - ↳ open each PC → Desktop → IP configuration → Static
   - * PC1: IP 192.168.1.11 | subnet 255.255.255.224 | Gateway 192.168.1.1
   - * PC2: IP 192.168.1.12 | subnet 255.255.255.224 | Gateway 192.168.1.1

5. Testing

* open Command prompt on PC1 and Test connection:

ping 192.168.1.1.





Result:

classless IP subnetting (/27 mask) was implemented successfully in Cisco Packet Tracer, and network communication was verified using the ping command.

Ex No: 10
a)

22/8/26

10a) Internetworking with Routers

Aim:
To connect two PCs in different networks using a router in Cisco Packet Tracer and verify connection using PDU packet.

procedure:

step 1: set up Devices

* Add : Router and 2PCs (PC0, PC1) to the work space.
* connect them using Copper - through straight cables.

step 2: configure PCs

* PC0 -> IP address : 192.168.168.10.2
  subnet Mask : 255.255.255.0
  Default Gateway : 192.168.10.1
* PC1 -> IP address : 192.168.20.2
  subnet mast : 255.255.255.0
  Default gateway : 192.168.20.1

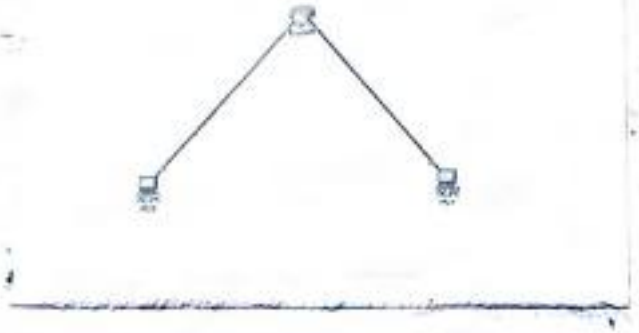
step 3: Configure the router

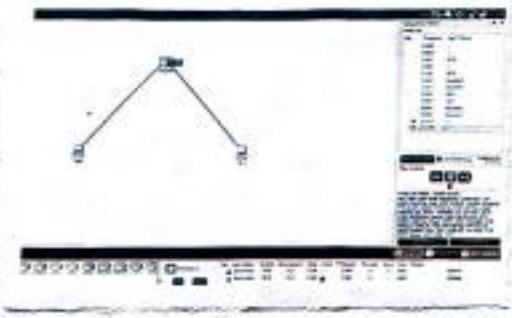
```
enable
config t
interface FastEthernet 0/0
ip address 192.168.10.1 255.255.255.0
no shut down.
exit
interface FastEthernet 0/1
ip address 192.168.20.1 255.255.255.0
no shut down
exit.
```

step 4: Test Connection

* click PC0 -> command promt -> type ping 192.168.20.2.

2. click add the simple PDU → click PC0 → click PC1 to see the successfull packet Transfer.





Result:

The router network was successfully created and tested by sending a PDU from PC0 to PC1.

Ex No: 10 b)

27/8/26

10. Internetworking with wireless Router, DHCP server, and Internet cloud.

Aim:

To design and configure, and test and internetwork using a wireless router, laptop, PC, cable, modem, Internet cloud, and a DHCP/DNS server in Cisco packet Tracer.

procedure:

step 1: Connect the topology

* place the devices in the workspace and connect them using the correct cables.

* PC → wireless router * wireless router → cable modem

* cable modem → Internet cloud * Internet cloud → Cisco.com server.

step 2: Configure the wireless router

1. click the wireless router → GUI Tab → wireless tab

* set network name (SSID): Home network

2. Go to the setup tab

* Ensure DHCP server is enabled.

* set static DNS 1: 208.67.220.220

step 3: Configure the laptop & PC

* click laptop → physical tab → Turn off power switch.

step 4: configure the Internet cloud.

step 5: configure the Cisco.com server.

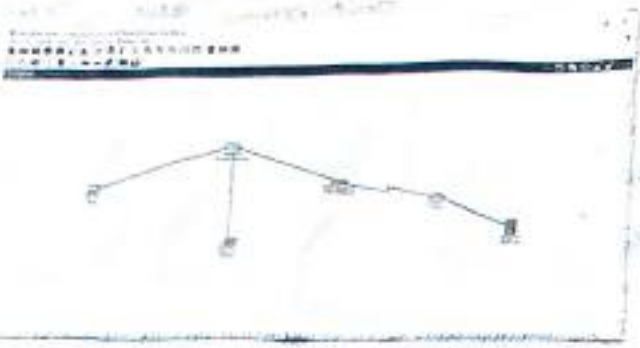
* service configuration

* static IP configuration

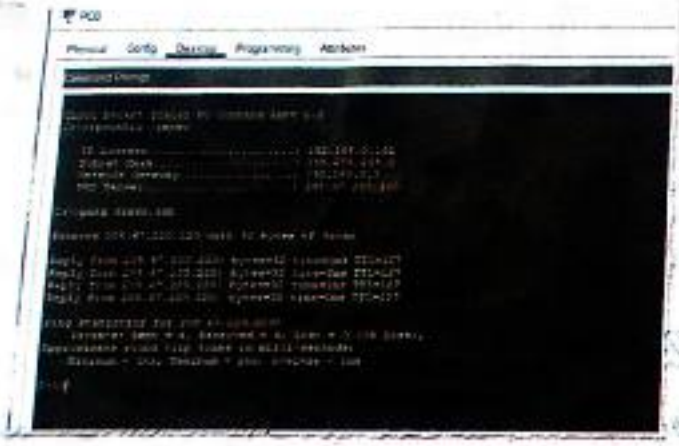
step 6: Test connectivity.

* Run ipconfig / renew to fetch dynamic.

* Run ping Cisco.com to confirm domain connectivity.







Result:

The network with topology, router, cable modem, Internet cloud, and DHCP/DNS server was successfully built and configured. Connectivity was verified by resolving and pinging cisco.com from the PC.

Ex.No: 11a)

29/8/26

11 a) Simulate static Routing Configuration.

Aim:

To configure static routing with primary and backup routes in Cisco Packet Tracer and verify path selection and failover using Tracert.

procedure:

step 1: Build the Topology

* place 3 routers (Router 0, Router 1, Router 2), 2 switches, 4 PCs and 1 server to the workspace
* connect pcs / server to switches using copper straight through cable.
* Connect switches to routers using copper straight through cable.
* connect routers to using crossover / serial cables.

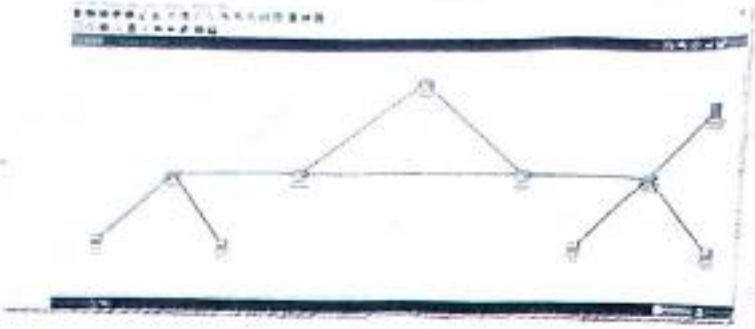
step 2: Assign Ip addresses on PCs & server

*Go to Desktop → Ip Configuration on each device.
* Assign Ip address, Mask and Gateway.

step 3: Configure the Router Interfaces.

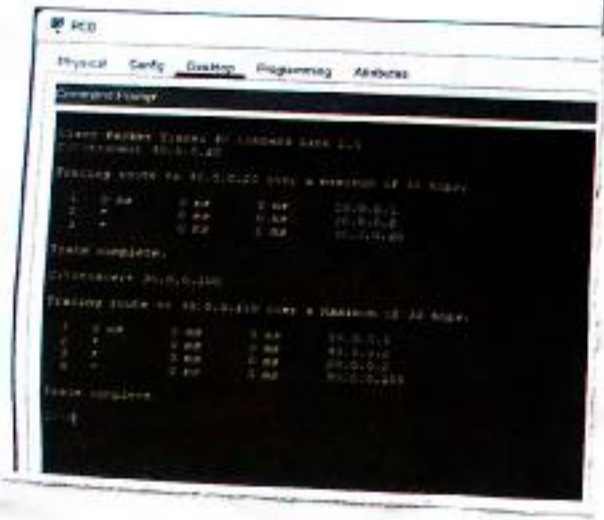
* Router 0

```
enable
config t
interface Gig 0/0
ip address 10.0.0.1 255.0.0.0
no shut down
interface Gig 0/1
ip address 20.0.0.1 255.0.0.0
no shutdown
interface Gig 0/2
ip address 40.0.0.1 255.0.0.0
```









* Router 1 : same for router 0.

enable

config

step 4 : Configure static Routers

step 5 : Verification & Testing

* check route : Go to pc0 → command prompt →

Type tracert 30.0.0.10 The packet goes through 20.0.0.2.

* Failover Test : Delete the cable b/w router 0 and router1

Result :

static routing with primary and backup routers was successfully configured and tested using Cisco Packet Tracer.


PC1
Physical
Config
Desktop
Programming
Attributes
Command Prompt
Cisco Packet Tracer PC Command Line 1.0
C:\>ipconfig
FastEthernet0 Connection:(default port)
Connection-specific DNS Suffix..:
Link-local IPv6 Address.....: FE80::2E0:F7FF:FE95:8171
IPv6 Address.....: ::
IPv4 Address.....: 10.0.0.20
Subnet Mask.....: 255.0.0.0
Default Gateway.....: ::
10.0.0.1
Bluetooth Connection:
Connection-specific DNS Suffix..:
Link-local IPv6 Address.....: ::
IPv6 Address.....: ::
IPv4 Address.....: 0.0.0.0
Subnet Mask.....: 0.0.0.0
Default Gateway.....: ::
0.0.0.0
C:\>ping 10.0.0.2
Pinging 10.0.0.2 with 32 Bytes of data:
Request timed out.
Request timed out.
Request timed out.
Request timed out.
Ping statistics for 10.0.0.2:
Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
C:\>

11 b)
10/9/26

Cisco packet Tracer

Aim:

To design, configure and simulate Routing Information protocol (RIP) in Cisco packet Tracer to enable dynamic routing and verify automatic traffic rerouting.

procedure:

step 1: Set up network Devices & connection

* Add 3 routers to the work place.
* connect end devices to routers using copper straight through cables.
* Interconnect routers using serial DCE cables.

step 2: Configure IP addresses on End devices

* click each PC → Desktop → IP Configuration and assign static IP parameters.
* Assign IP address subnet mask, Default Gateway for each end devices.

step 3: Configure the Router Interfaces & IP Address.

* open cli tab on the each router and run the interface configuration commands.

step 4: Configure RIP protocol on Routers.

* Enable each RIP on each router and advertise directly connected networks IPs:

step 5: Testing & Verification

* ping Test: open pc0 → command prompt and test connectivity ping 20.0.0.2.

* Trace Route Test: Test Trace the path to verify that RIP selects the lowest hop count route.

* Dynamic re Routing Testing.

Result:

RIP routing protocol was configured and verified. Dynamic rerouting during link failure was confirmed using.

Ex No 12a) Echo client - server using TCP sockets

12a)
29/9/26

Aim:

To write and execute a c/ Java socket program to implement a TCP Echo client and echo server.

Algorithm:

server side:

1. create a socket using socket().
2. Bind the socket the ~~IP~~ address and port number using bind()
3. Listen for incoming client connecting using listen().
4. Accept the connection request from the client using accept().
5. Read / Receive the message send by the client.
6. send / Echo the same message back to the client
7. close the client and server sockets.

client side:

1. create socket using socket().
2. connect to the server using connect().
3. Take input (message) from the user.
4. send the message to the server.
5. Receive / Read the echoed message back from the server.
6. Display the echoed message on the screen.
7. close the client socket.

program code:

EchoServer.java

```
import java.io.*;
import java.net.*;
public class EchoServer{
    public static void main (String[] args){
        try{
```

```
ServerSocket serverSocket = new ServerSocket(5000);
System.out.println("Echo server started. waiting for client...");
Socket socket = serverSocket.accept();
System.out.println("Client connected!");
BufferedReader in = new BufferedReader(new InputStreamReader(socket.
                                    getInputStream()));
PrintWriter out = new PrintWriter(socket.getOutputStream(),
                                    true);
String inputLine = in.readLine();
System.out.println("Received from client: " + inputLine);
out.println(inputLine);
System.out.println("Echoed back to client.");
socket.close()
serverSocket.close();
} catch (IOException e){
    e.printStackTrace();
}
}
}
```

EchoClient.java

```
import java.io.*;
import java.net.*;
public class EchoClient {
    public static void main(String[] args){
        try{
            Socket socket = new Socket("localhost", 5000);
            System.out.println("Connected to Echo Server!");
        BufferedReader userInput = new BufferedReader(new InputStream
                                    Reader(System.in));
        BufferedReader in = new BufferedReader(new InputStreamReader
                                    (socket.getInputStream()));
        PrintWriter out = new PrintWriter(socket.getOutputStream
                                    (), true);
        System.out.print("Enter message to Echo:");
        String message = userInput.readLine();
        out.println(message);
        String echoResponse = in.readLine();
```

```
            System.out.println("Server Echo Response: " +
                echoResponse);
            socket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Output:

server output:

Echo server started, waiting for client
client connected.
Received from client: Hello computer Networks Lab
Echoed back to client.

client output:

Connected to Echo server!
Enter message to Echo: Hello computer network Lab
Server Echo Response: Hello computer network Lab.

Result:

The TCP Echo client/server program was successfully implemented, compiled, and executed and the server correctly echoed the received message back to the client.

Ex.No : 12b
17/9/24

12) b chat client - Server using Tcp sockets

Aim:
To write and execute a c/java socket program to implement a two way chat client and chat server and the server correctly echoed the received message back to the client.

Algorithm:
Server side:
1. create a socket and bind it to a designed port.
2. Listen and accept connection from the client.
3. start the communication loop
4. close the socket when chat ends.

client side:
1. create a socket and connect to the server IP and port.
2. start a communication loop.
3. close the socket connection.

program code:
chatServer.java

```
import java.io.*;
import java.net.*;
public class ChatServer{
    public class static void main(String[] args){
    try{
      ServerSocket serverSocket = new ServerSocket(6000);
      System.out.println("Chat Server running on port 6000..
                          waiting for client...");
      Socket socket = serverSocket.accept();
      System.out.println("Client connected! start chatting");
      BufferReader in = new BufferReader(new InputStreamReader
                                          (socket.getInputStream()));
      PrintWriter
      BufferReader out = new PrintWriter(socket.getOutputStream(),
                                          true);
      BufferReader userIn = new BufferReader(new InputStreamReader
                                          (System.in));
```

```
String clientMsg = "", serverMsg = "";
while (!clientMsg.equalIgnoreCase("stop")) {
    clientMsg = in.readLine();
    if (clientMsg == null clientMsg.equalIgnoreCase("stop")) {
        System.out.println("Client disconnected the chat");
        break;
    }
    System.out.println("Client: " + clientMsg);
    System.out.print("Server: ");
    serverMsg = sysIn.readLine();
    out.println(serverMsg);
}
serverSocket.close();
serverSocket.close();
} catch (IOException e) {
    e.printStackTrace();
}
}
```

chatClient.java

```
import java.io.*;
import java.net.*;
public class chatClient {
    public static void main(String[] args) {
        try{
            Socket socket = new Socket("127.0.0.1", 6000);
            System.out.println("Connected to chat server! Type 'stop' to exit.");
        BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        BufferedReader sysIn = new BufferedReader(new InputStreamReader(System.in));
        String clientMsg = "", serverMsg = "";
        while (!clientMsg.equalIgnoreCase("stop")){
            System.out.print("Client: ");
            clientMsg = sysIn.readLine();
            out.println(clientMsg);
            if (clientMsg.equalIgnoreCase("stop")) {
                break;
            }
            serverMsg = in.readLine();
            System.out.println("Server: " + serverMsg);
        }
        socket.close();
    } catch (IOException e) {
```

}
}
}

Output:

chat server running on port 6000... waiting for client...
client connected! start chatting.
Client: Hi server, how are you?
server: Hello client! I am fine
Client: connection working fine
server: Great! Bye.
client: stop
client: disconnected the chat.

client Console

connected to chat server! Type 'stop' to exit.
client: Hi server, how are you?
server: Hello client! I am fine.
client: Connection working fine.
Server: Great! Bye
client: stop.

Result:

The TCP chat client server program was successfully written, compiled, and executed, allowing real time two-way communication b/w client & server.

24/9/26

Aim:

To write and exute a simple java prognam implement a custom ping application using java InetAddress class and measure network reachability a round_trip time.

Alogorithm:

1. Get Target Host: Accept hostname or ip address input from the user.
2. Resolve Host Address: Use InetAddress.getByName() to resolve the host.
3. Record start Time: save the current system time before sending the request.
4. send Echo ping: Call address.isReachable(timeout) to send an ICMP Echo request.
5. Calculate RTT: Record the end time and compute Round_trip Time (RTT = End Time_start Time)
6. Display Result: Print whether the host is reachable along with the response time in milliseconds.

program code:

```
import java.net.InetAddress;
import java.util.Scanner;
public class pingProgram{
    public static void main(String[] args){
        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter host to ping :");
        String host = scanner.nextLine();
        try{
            InetAddress address = InetAddress.getByName(hos
            System.out.println("\n Pinging "+ host + "["+ address.
                                getHostAddress()+"]");
```

```
            int timeout = 3000;
            long startTime = System.currentTimeMillis();
            boolean reachable = address.isReachable(timeout);
            long endTime = System.currentTimeMillis();
            long rtt = endTime - startTime;
            if(reachable){
              System.out.println("Reply From " + address.getHostAddress() +
                                 ": Time =" + rtt + "ms");
            } else {
              System.out.println("Request Timed out.");
            }
          } catch (Exception e) {
            System.out.println("Cannot resolve host or ping error: "
                               + e.getMessage());
          } finally {
            scanner.close();
          }
        }
      }
```

output:

```
pinging 127.0.0.1 [127.0.0.1]:
Reply from 127.0.0.1 : Time = 1ms.
```

Result:

The custom ping program in Java was successfully executed, and host reachability along with the round trip response time was verified.

Ex No: 14
24/9/26

Network Interface Information in Java

Aim:

To write and execute a simple java program to retrive local network interface details and assigned Ip addresses.

Algorithm:

1. Call NetworkInterface.getNetworkInterfaces() to get all network interfaces.
2. Loop through each network interface using collection list()
3. print the interface name and display name.
4. Get all Ip address linked to each interface using getInetAddresses() and print them.

program:

```
import java.net.InetAddress;
import java.net.NetworkInterface;
import java.util.Collections;
public class SimpleNetworkInfo {
    public static void main(String[] args){
    try{
      for(NetworkInterface netIf : Collections.list(NetworkInterface.
                                    getNetworkInterfaces())){
      System.out.println("Interface: " netIf.getName() + " ("+ netIf.
            getDisplayName() + ")");
      for (InetAddress addr : Collection.list(netIf.getInetAddress
                                                    ())){
          System.out.println("  -> Ip: " + addr.getHostAddress());
      }
      System.out.println();
    }
    } catch (Exception e){
        System.out.println(" Error: " + e.getMessage());
    }
    }
}
```

Output:

Interface: lo (Software Loopback Interface 1)

→ IP: 127.0.0.1

→ IP: 0:0:0:0:0:0:0:1

Interface: echo (Realtek PCIe GbE Family Controller)

→ Ip: 192.168.1.10

Result:

The java network interface program was successfully compiled and executed, displaying the available network interfaces and their assigned Ip addresses.

8/10/26

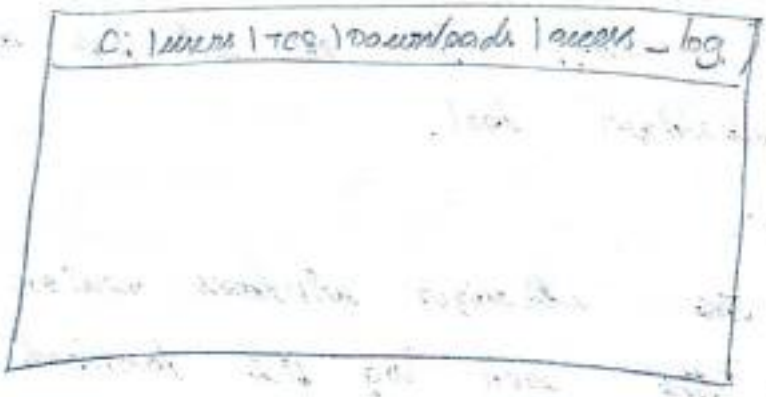
Aim:

To analyz types of web logs using the webalizer tool.

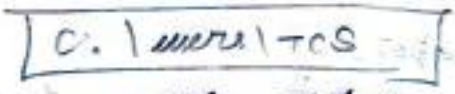
procedure:

1. Run the webalizer windows version
2. Input the web log file download from the web
3. press Run webalizer.
4. ~~web~~ webalizer processes the log file and generates statistical reports
5. view the generated report in web browser.
6. ~~web~~ webalizer reads and process the log file.
7. It analyses the requests and ~~vi~~ session information.
8. The tool generates statical reports and graphs.
9. open the generated report in a web browser
10. Analyze the number of visits, pages and request, bandwidth usage.

log files.

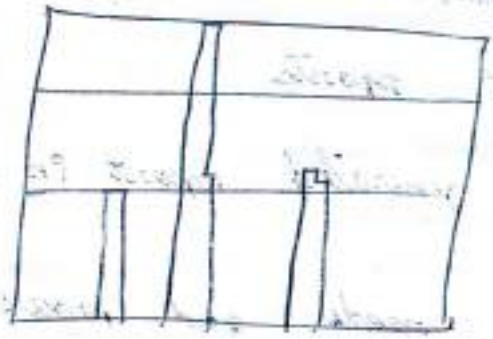


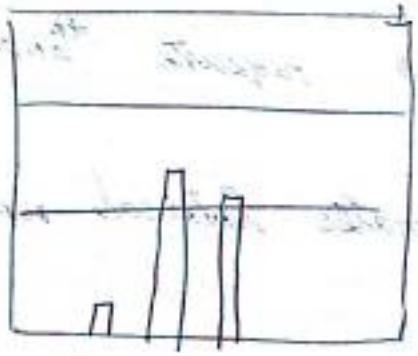
Target directory



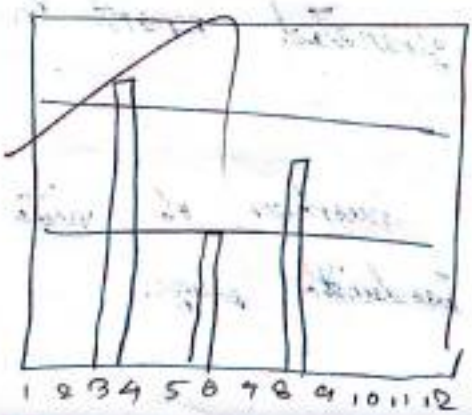
output: Monthly statistics

599





4509



Result :

Thus the different types of web logs were succesfully analyzed using the webalizer tool.